

CS 4150 ALGORITHMS

Spring 2026

Assignment 1: Algorithm Analysis

Due Date: 11:59 p.m., Tuesday, January 27, 2026

Submission instructions (applicable to all assignments in this semester)

- Please review the homework guidelines on the syllabus.
- Turn in your homework on Gradescope (you may access it through Canvas: Once you log into Canvas, there is a “Gradescope” tab on the left side, click it, and then follow the instructions). Please submit it on time. Gradescope will be **automatically closed** at the due time.
- Upload **a single file** with all problems to Gradescope. When Gradescope prompts you, **assign the pages** that contain each problem (this will facilitate grading). If you upload several files, the last upload will override all previous uploads, so do **not** upload individual problems separately.
- Although not required, you are strongly encouraged to **typeset your solutions** (e.g., using Word or latex; the latex source file of this homework file can be found on Canvas in hw1 folder). If you still opt for handwriting, then make sure everything is clear. Remember that you could **lose points** only because your handwriting is illegible.
- If you have questions about homework, feel free to ask TA. However, TA will **not “pre-grade”** your homework, which means that you are not supposed to bring your solution to TA and simply ask whether your solution is correct.

Note: If not specified, the base of log is 2. This applies to all assignments in this semester.

1. **(10 points)** This exercise is to convince you that exponential time algorithms should be avoided if possible.

Suppose we have an algorithm A whose running time is $O(2^n)$. For simplicity, we assume the algorithm A needs 2^n instructions to finish, for any input size of n (e.g., if $n = 5$, A will finish after $2^5 = 32$ instructions). (You may consider an instruction a basic computation step.)

According to Wikipedia, as of June 2025, the fastest supercomputer in the world is the United States’ EI Capitan (hosted at the Lawrence Livermore National Laboratory in Livermore, California) and can perform about 1742×10^{15} instructions per second.

Suppose we run the algorithm A on EI Capitan. Answer the following questions.

- (a) For the input size $n = 100$, how much time does EI Capitan need to finish the algorithm? Give the time in terms of **centuries**.
- (b) For the input size $n = 1000$, how much time does EI Capitan need to finish the algorithm? Give the time in terms of **centuries**.

Note: For simplicity, you may assume that a year always has exactly 365 days. For both questions, an approximate answer is fine as long as it is not too far from the “accurate” answer.

2. **(20 points)** Order the following list of functions in asymptotically increasing order (i.e., from small to large).

$$\begin{array}{cccccc} \log n & n! & 2^{500} & 2^n & \log(\log n)^2 & 2^{\log n} \\ \log^3 n & n \log n & \log_4 n & n^3 & \sqrt{n} & n^2 \log^5 n \end{array}$$

3. **(30 points)** For each of the following pairs of functions, indicate whether it is one of the three cases: $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, or $f(n) = \Theta(g(n))$. For each pair, you only need to give your answer, and the proof is NOT required.

- (a) $f(n) = 9 \log n$ and $g(n) = \log n^4 + 6$.
- (b) $f(n) = n^2 + n \log^3 n$ and $g(n) = 6n^3 + \log^2 n$.
- (c) $f(n) = 5^n$ and $g(n) = n^2 2^n$.
- (d) $f(n) = n \log^2 n$ and $g(n) = \frac{n^2}{\log^3 n}$.
- (e) $f(n) = \sqrt{n} \log n$ and $g(n) = \log^8 n + 25$.
- (f) $f(n) = n \log n - 6n$ and $g(n) = n \log_5 n + 8n$.

4. **(20 points)** This is a “warm-up” exercise on algorithm **design** and **analysis**.

The *knapsack problem* is defined as follows: Given as input a knapsack of size K and n items whose sizes are $k_1, k_2, \dots, k_n$, where K and $k_1, k_2, \dots, k_n$ are all **positive real numbers**, the problem is to find a full “packing” of the knapsack, that is, choose a subset of the n items such that the total sum of the sizes of the items in the chosen subset is *exactly* equal to K .

It is well known that the knapsack problem is NP-complete, which implies that efficient algorithms (i.e., those with a polynomial running time) for this problem are unlikely to exist. Therefore, people tend to look for good **approximation algorithms** to solve this problem. In this exercise, we relax the constraint of the knapsack problem as follows.

We still seek a packing of the knapsack, but we need not look for a “full” packing of the knapsack; instead, we look for a packing of the knapsack (i.e., a subset of the n input items) such that the total sum of the sizes of the items in the chosen subset is *at least* $K/2$, but no more than K . This is called a *factor of 2 approximate solution* for the knapsack problem. To simplify the problem, we assume that a factor of 2 approximate solution for the knapsack problem always exists, i.e., there always exists a subset of items whose total size is at least $K/2$ and at most K .

For example, if the sizes of the n items are $\{9.3, 24, 14, 5, 8, 17, 4.2\}$ and $K = 20$, then $\{9.3, 5, 4.2\}$ is a factor of 2 approximate solution. Note that such a solution might not be unique. For example, $\{14\}$ is also a solution and $\{8, 4.2\}$ is also a solution.

Design a **polynomial time** algorithm for computing a factor of 2 approximate solution, and analyze the running time of your algorithm (using the big- O notation). Note that, although there may be multiple solutions, your algorithm only needs to find one solution.

If your algorithm runs in $O(n)$ time and is correct, then you will get **5 bonus points**.

Algorithm design problem guideline: To answer algorithm design problems like Question 4, please follow the following guideline, which applies to the algorithm design questions in *all assignments* in this semester.

1. **Algorithm Description** You are asked to clearly describe the main idea of your algorithm.

- 2. Pseudocode** You are required to provide pseudocode. Pseudocode is an effective tool for explaining algorithms. If your pseudocode clearly conveys the main idea of your algorithm, then a separate algorithm description is not required. Note that *you should not use actual Python, Java, or C++ code as “pseudocode,”* since real programming code is often difficult to read at a conceptual level. In principle, pseudocode should be easy to read while explicitly capturing all essential operations of the algorithm, as demonstrated in class.
- 3. Correctness** Unless in some cases where the correctness is obvious, you need to (at least briefly) explain why your algorithm is correct. For instance, for this knapsack problem, you need to explain why the solution produced by your algorithm is a factor of 2 approximate solution.
- 4. Time Analysis** You are also required to analyze the time of your algorithm. For example, for this knapsack problem, if you claim the runtime of your algorithm is $O(n)$, then you need to explain why it is $O(n)$. *Please be honest:* If you know that the time of your algorithm is not $O(n)$, you should not attempt to claim it.

Total Points: 80 (not including the 5 bonus points)